

LPJS - Lightweight, Portable Job Scheduler

Contents

1	Introduction	1
2	Installation	2
3	Configuration	3
4	Starting Daemons	4
4.1	General Info	4
4.2	Daemons as a Service	4
4.3	Ad hoc Clusters and Grids	4
5	Network topology	6
6	File Sharing	7
7	Advanced configuration	9

Chapter 1

Introduction

LPJS is a resource manager and job scheduler for running batch jobs on one or more computers. It can be used on a single machine in order to maximize utilization of CPUs and memory without oversubscribing the system, or on multiple networked computers organized as an HPC (high performance computing) cluster or HTC (high throughput computing) grid.

Unlike most similar tools, LPJS is designed to be simple, easy to install and configure, easy to use, and portable to any POSIX platform. It provides an intuitive user interface, including menu-based operation for most common tasks.

This manual is aimed at the systems manager, covering installation and configuration of LPJS. For a user's guide, including general information on HPC clusters and HTC grids, see the Research Computing User's Guide at <https://acadix.biz/publications.php>.

Chapter 2

Installation

An HPC cluster or HTC grid is a group of computers including a *head node*, which manages the cluster or grid, and one or more *compute nodes*, which run the computational software. LPJS must be installed and configured on the head node and all compute nodes.

Installation should be performed using a package manager, such as FreeBSD ports or [dreckly](https://github.com/outpaddling/dreckly). We maintain a FreeBSD port for use on FreeBSD and Dragonfly BSD, and a dreckly package that should work on any other POSIX platform, including other BSDs, most Linux distributions, macOS, Solaris-based systems, etc. Other package managers may be supported by third parties. If you would like to add LPJS to your favorite package manager, see the instructions for packagers in the README at <https://github.com/outpaddling/LPJS/>.

The dreckly package manager can be quickly and easily installed on most POSIX platforms using the [auto-dreckly-setup](#) script. Simply download the script to your Unix computer, run **sh auto-dreckly-setup** in a terminal window, and follow the instructions on the screen.

LPJS uses **munge** (<https://github.com/dun/munge>) to encrypt and authenticate messages between nodes. Munge is installed automatically by the package manager when installing LPJS, and automatically configured by LPJS administration scripts.

Munge requires all nodes to have a shared munge key file, which is unique to your installation. It must be generated by you and distributed to all computers that are part of your cluster or grid. **THE MUNGE KEY FILE MUST BE KEPT SECURE AT ALL TIMES ON ALL NODES.** Use secure procedures to distribute it to all nodes, so that it is never visible to unauthorized users. The **lpjs admin** command provides a menu with an option for enabling the munge daemon. This option can securely copy the munge key from another computer.

Chapter 3

Configuration

LPJS is designed to require minimal configuration. For example, compute node resources such as processors and memory are determined automatically when the **lpjs_compd** daemon starts and need not be specified in configuration files.

Most configuration can be done entirely using **lpjs_admin** (**man lpjs_admin**), a menu-driven admin tool. Simply run **lpjs_admin**, select an item from the menu, and answer the questions on the screen. For the sake of understanding what **lpjs_admin** does, some basic information is provided below.

The head node runs the *dispatch daemon*, **lpjs_dispatchd**, which keeps track of all computing resources (e.g. processors, memory) on the compute nodes and dispatches jobs to compute node with sufficient available resources. The head node requires a configuration file, which in its simplest form merely lists the complete host names (FQDNs) of the head node and each compute node, one node per line, e.g.

```
head    myhead.mydomain
compute compute001.mydomain
compute compute002.mydomain
...
```

The FQDN (fully qualified domain name) must match the name reported by running **hostname** on that node. On other nodes, this is either listed in `/etc/hosts` or provided by *DNS* (*domain name service*).

Each compute node runs the *compute daemon*, **lpjs_compd**. Hardware specs for compute nodes are automatically determined by **lpjs_compd** on the compute nodes and reported to **lpjs_dispatchd** on the head node. They may be overridden in the configuration file on the head node in order to reserve memory or processors for non-LPJS use, e.g.:

```
head    myhead.mydomain
compute compute001.mydomain
# compute-002 actually has 16GiB RAM and 8 processors, but
# we limit LPJS to half of each
compute compute002.mydomain pmem=8GiB processors=4
...
```

Note Overrides for processors and pmem in the configuration file are ignored if they exceed the auto-detected specs reported by **lpjs_compd**.

Each compute node requires the same type of configuration file, but it need only list the head node. It can be a copy of the configuration file used on the head node, in which case the compute node entries are ignored. You may wish to create a configuration file on the head node and simply distribute it.

Chapter 4

Starting Daemons

4.1 General Info

All nodes in the cluster or grid must be running **munged**, using the same munge key. The head node must also run **lpjs_dispatchd**, and all compute nodes must run **lpjs_compd**.

Appropriate services can be configured by running **lpjs admin** on each node.

The head node can also serve as a compute node, though this is not generally recommended. The head node of most clusters and grids should remain lightly loaded so that it can respond promptly to events that occur such as new job submissions and job completions. The head node need not be a powerful machine. A laptop or low-end desktop machine will work just fine for a small cluster. Laptops are actually nice in that they have a built-in battery backup, so your head node at least is protected against brief power outages.

The head node on larger clusters need not be powerful either, but should be highly reliable. We recommend a server with a mirrored boot disk, hot-swap disks, redundant power supplies, also hot-swap, and a UPS (uninterruptable power supply). With this hardware configuration, down time should be near zero. The FreeBSD installer makes it trivial to mirror a boot disk using ZFS any computer with two drives. However, a hardware RAID card will make it easier to swap out a bad disk than a ZFS RAIDZ, assuming the disks are hot-swap. With a quality hardware RAID card, we can generally just remove the bad disk, replace it with an equivalent one, and the RAID card will automatically configure the new disk.

Jobs can be submitted, monitored, and controlled from any computer with a compatible version of LPJS and an appropriate configuration.

Note

A computer need not even be running LPJS daemons, but it does require an LPJS configuration file listing the head node, a running munge daemon, and the same munge key as the other nodes.

Computers configured to interact with an LPJS cluster/grid without running LPJS daemons are called "UI" (user interface) nodes.

4.2 Daemons as a Service

The **lpjs_dispatchd** and **lpjs_compd** commands are normally run as a service, which automatically starts when the computer is rebooted. You can run **lpjs admin** and use the menus to configure a machine as a head node or compute node with the appropriate services enabled. This will require administrative rights on each computer in the cluster/grid.

4.3 Ad hoc Clusters and Grids

It is also possible to use LPJS without enabling services, even without having admin rights. Simply start the daemons manually by running **munged** and **lpjs_dispatchd** on the head node, and **munged** and **lpjs_compd** on each compute node.

Note Note that if **lpjs_compd** is not running as root, the compute node will only be able to run jobs under the same user name running **lpjs_compd**.

The **lpjs ad-hoc** command displays a menu, allowing you to start and stop the appropriate daemons without having to know the precise commands, with or without administrative rights on the computer.

Chapter 5

Network topology

A *cluster* is generally a collection of dedicated computers, all connected directly to the same private, often high speed network. In this sense, a cluster is a LAN (local area network). In many cases, special network technology, such as *Infiniband*, is used in place of, or in addition to, standard Ethernet. Infiniband and similar technologies offer much lower latency per message, and higher throughput.

A *grid* is conceptually like a cluster, in that it is used for distributed parallel computing (parallel computing across multiple computers). However, grids are more loosely coupled, often utilizing computers that are not on a dedicated LAN, and possibly not even in the same location. Hence, a grid is not as suitable for parallel computing that involves a lot of communication between processes, such as MPI (Message Passing Interface) distributed parallel programs.

LPJS is very flexible with network topology. The *only* requirement is that all nodes are able to connect to the head node. This means that a cluster or grid using LPJS can consist of other computers on the same LAN, computers in different buildings, virtual machines behind a *NAT* (*network address translation*) firewall, or cloud instances in a data center a thousand miles away. You must consider how each of these resources can effectively be used, though. Network latency and throughput may be quite poor for resources that are far away.

Chapter 6

File Sharing

Clusters normally have one or more file servers, so that jobs can run in a directory that is directly accessible from all nodes. This is the ideal situation, as input files are directly available to jobs, and output files from jobs can be written to their final location without needing to transfer them.

Note

At present, it appears to be impractical to use macOS for compute nodes with data on a file server. macOS has a security feature that prevents programs from accessing most directories unless the user explicitly grants permission via the graphical interface. In order for LPJS to access file servers as required for normal operation, the program **lpjs_compd** must be granted full disk access via System Settings, Privacy and Security. Otherwise, you may see "operation not permitted" errors in the log when trying to access NFS shares.

The major problem is that this is not a one-time setting. Each time LPJS is updated, full disk access is revoked, and the user must enable it via the graphical interface again.

Grids normally do not have file servers. In this case, it will be necessary for all nodes to have the ability to pull files from and push files to *somewhere*. Typically, this somewhere would be the submit node, or a server accessible for file transfers from the submit node and all compute nodes.

LPJS does not provide file transfer tools. There are numerous highly-evolved, general-purpose file transfer tools already available, so it is left to the systems manager and user to decide which one(s) to use. We recommend using **rsync** if possible, as it is highly portable and reliable, and minimizes the amount of data transferred when repeating a transfer.

Note All compute nodes must be able to perform a passwordless file transfers to the designated server, i.e. pulling files to, or pushing files from a compute node does not prompt the user for a password. This is generally accomplished by installing ssh keys on the submit node, which can be done by running **auto-ssh-authorize submit-host** from every compute node, as every user who will run jobs.

The **lpjs submit** command creates a marker file in the working directory on the submit host, named ".lpjs-submit-host-name-shared-fs-marker" (replace "submit-host-name" with the FQDN of your submit node). If this file is not accessible to the compute node, then LPJS will take the necessary steps to create the temporary working directory and transfer it back to the submit node after the script terminates.

If the working directory (the directory from which the job is submitted on the submit node) is not accessible to the compute nodes (e.g. using NFS), then the user's script is responsible for downloading any required input files. Below is an example from `Test/fastq-trim.lpjs` in the LPJS Github repository.

Note Note that we used the `--copy-links` option with `rsync`, so that it copies files pointed to by symbolic links, rather than just recreating the symbolic link on the compute node. You must understand each situation and decide whether this is necessary.

```
# Marker file is created by "lpjs submit" so we can detect shared filesystems.
# If this file does not exist on the compute nodes, then the compute nodes
# must pull (download) the input files.
marker=.lpjs-$LPJS_SUBMIT_HOST-shared-fs-marker
if [ ! -e $marker ]; then
    printf "$marker does not exist. Using rsync to transfer files.\n"
    set -x
    printf "Fetching $LPJS_SUBMIT_HOST:$LPJS_WORKING_DIRECTORY/$infile\n"
    # Use --copy-links if a file on the submit node might be a symbolic
    # link pointing to something that it not also being pulled here
    rsync --copy-links ${LPJS_SUBMIT_HOST}:$LPJS_WORKING_DIRECTORY/$infile .
    set +x
else
    printf "$marker found. No need to transfer files.\n"
fi
```

LPJS will, by default, transfer the contents of the temporary working directory back to the working directory on the submit node, using **rsync -av temp-working-dir/ submit-host:working-dir**. The "working-dir" above is the directory from which the job was submitted, and "temp-working-dir" is a job-specific temporary directory created by LPJS on the compute node. Following this transfer, the working directory on the submit node should contain the same output file as it would using a shared filesystem. Users can override the transfer command. command. See the Research Computing User Guide for details.

```
# If we downloaded the input file, remove it now to avoidwasting time
# transferring it back. By default, LPJS transfers the entire temporary
# working directory to the submit node using rsync.
if [ ! -e $marker ]; then
    rm -f $infile
fi
```

Chapter 7

Advanced configuration

TBD